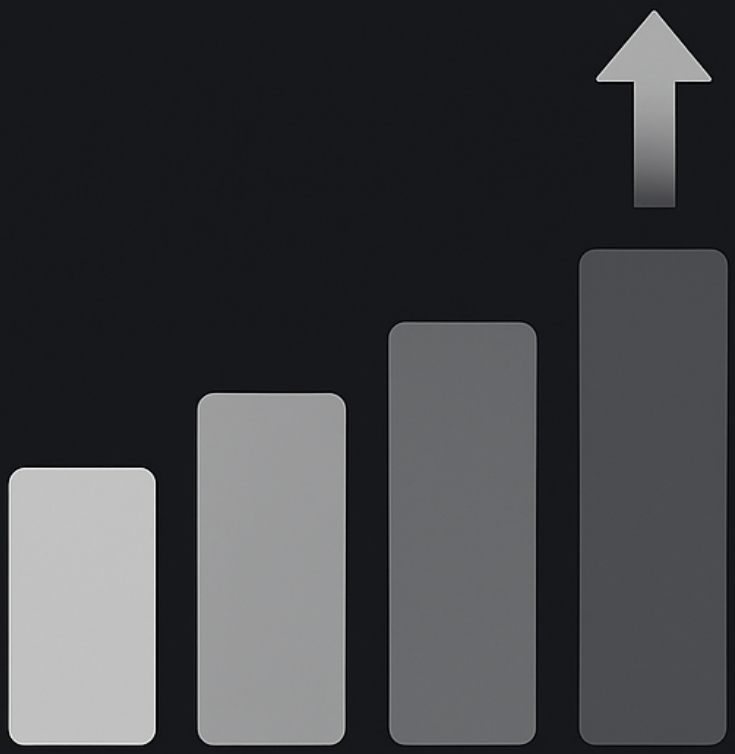
DATA STRUCTURES AND ALGORITHMS



SORTING

C {...}

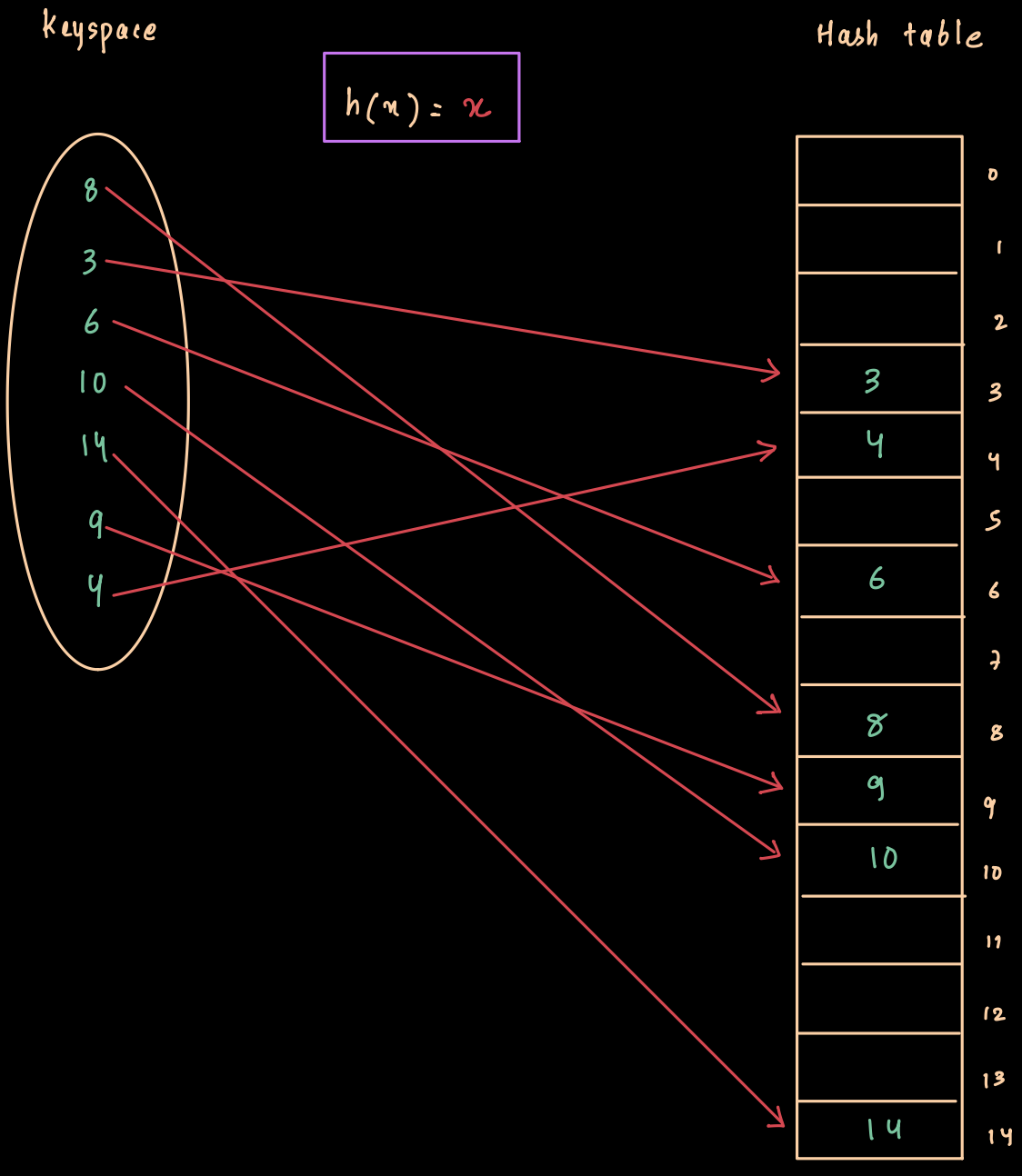
ai

365. Introduction to Hashing

Mappings : (I) One - One (Ideal)
(II) Many - One (Modified)
(III) One - Many
(IV) Many - Many

(i) One - One mapping ex :

keys : 8, 3, 6, 10, 14, 9, 4



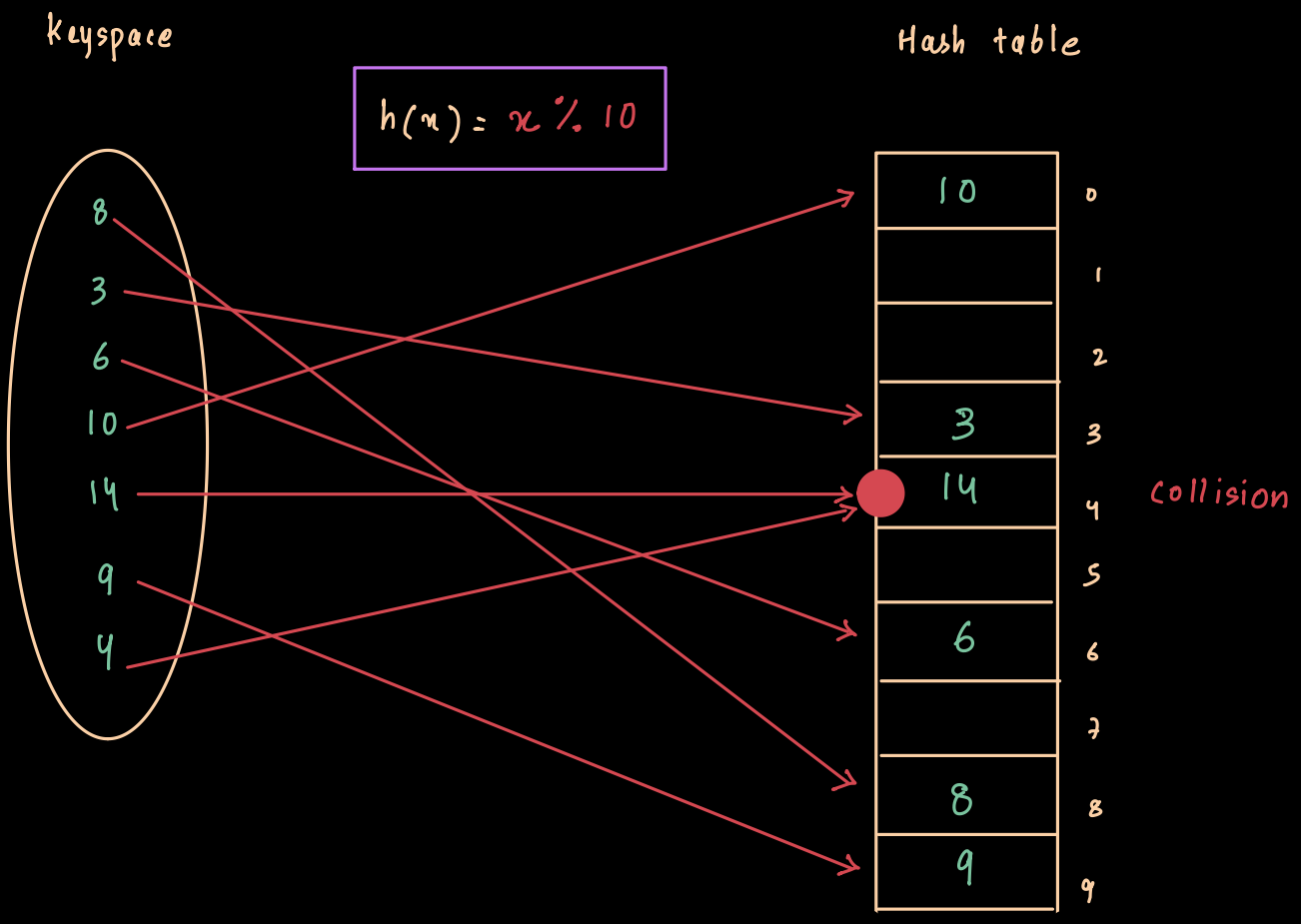
Drawback :- Space complexity ↑

↳ Hash function responsible

Advantage :- Faster → O(1)

(ii) Many – One ex : (Modified hashing)

keys : 8, 3, 6, 10, 14, 9, 4



Disadvantage : Collision has happened for 4 & 14.

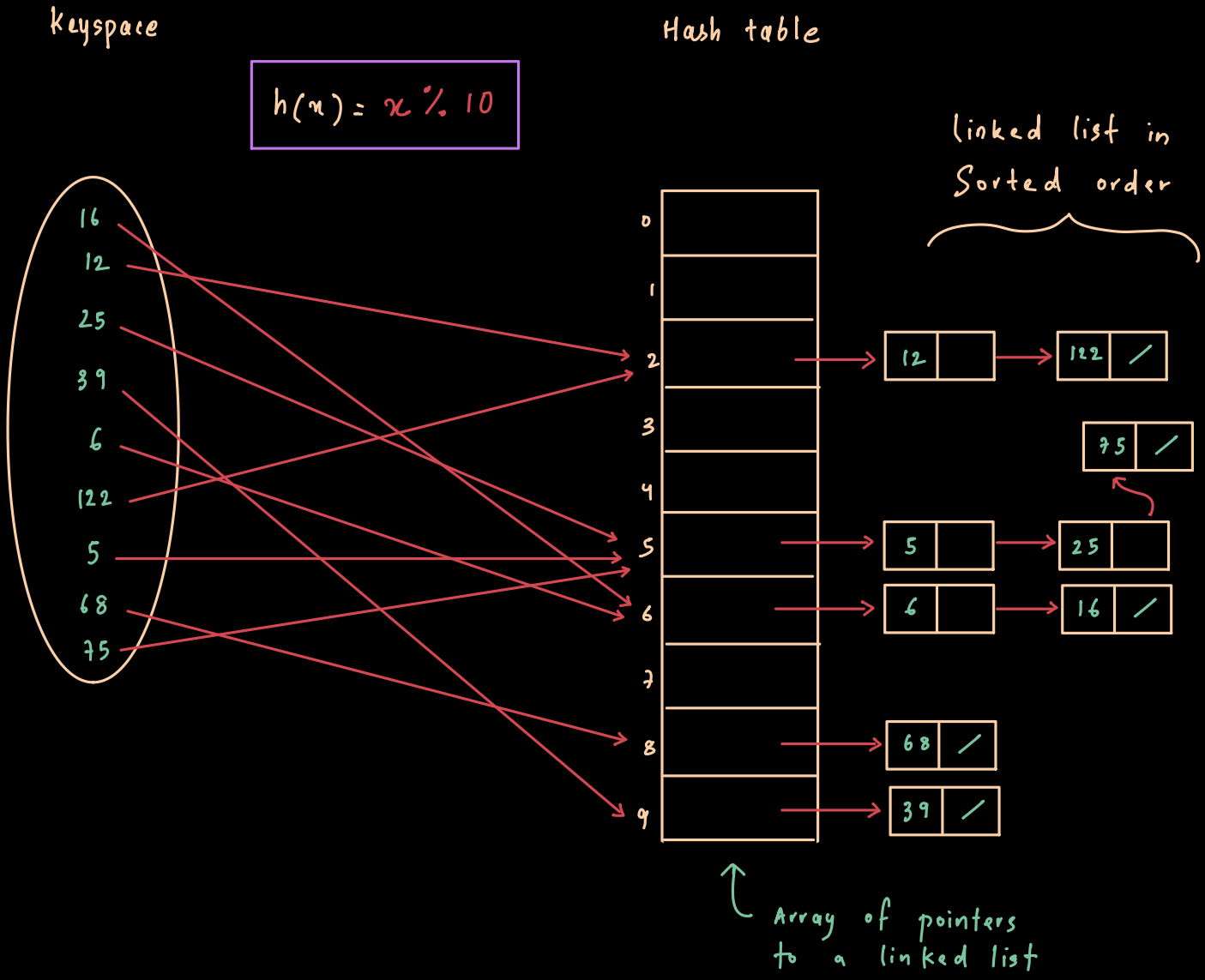
RESOLVING COLLISION :- (codes not req. for exam)

1) Open Hashing : - chaining

2) Closed Hashing : - linear probing
- Quadratic probing
- Double hashing

366. Chaining

keys : 16, 12, 25, 39, 6, 122, 5, 68, 75



* Loading Factor (λ) = $\frac{\text{no. of keys } (n)}{\text{size of Hash table}}$

↳ Analysis of Hashing is done using loading factor.

* Avg. Time taken for successful search :

$$t = 1 + \frac{\lambda}{2}$$

searching in Hash table

avg. Searching time in L.L

* Avg. Time taken for unsuccessful search :

$$t = 1 + \lambda$$

searching in Hash table

Searching time in L.L (searching till end)

* It is not necessary to always consider the last element in key.

ie; keys : 1(6) , 1(2) , 2(5) , 13(9)

We can also take the 2nd last element.

ie; keys : (1)6 , (1)2 , (2)5 , 1(3)9

Hash function :- $h(x) = (x/10) \% 10$

* Case : keys : 5, 15, 25, 35, 45, 55, 65, 75, 85, 95

$H(x) = x \% 10$

Now, have loading factor (λ) $= \frac{10}{10} = 1$

→ Acc. to loading factor, each key should be uniformly distributed.

→ But we can clearly see that the whole hash table is empty and only Index 4 is filled up (position 5)

→ So, Hashing isnot dependant upon loading factor, but HASH FUNCTION provided by the developer.

ie ; If hash function in this case was :

$h(x) = (x/10) \% 10$

→ All keys could be uniformly distributed.

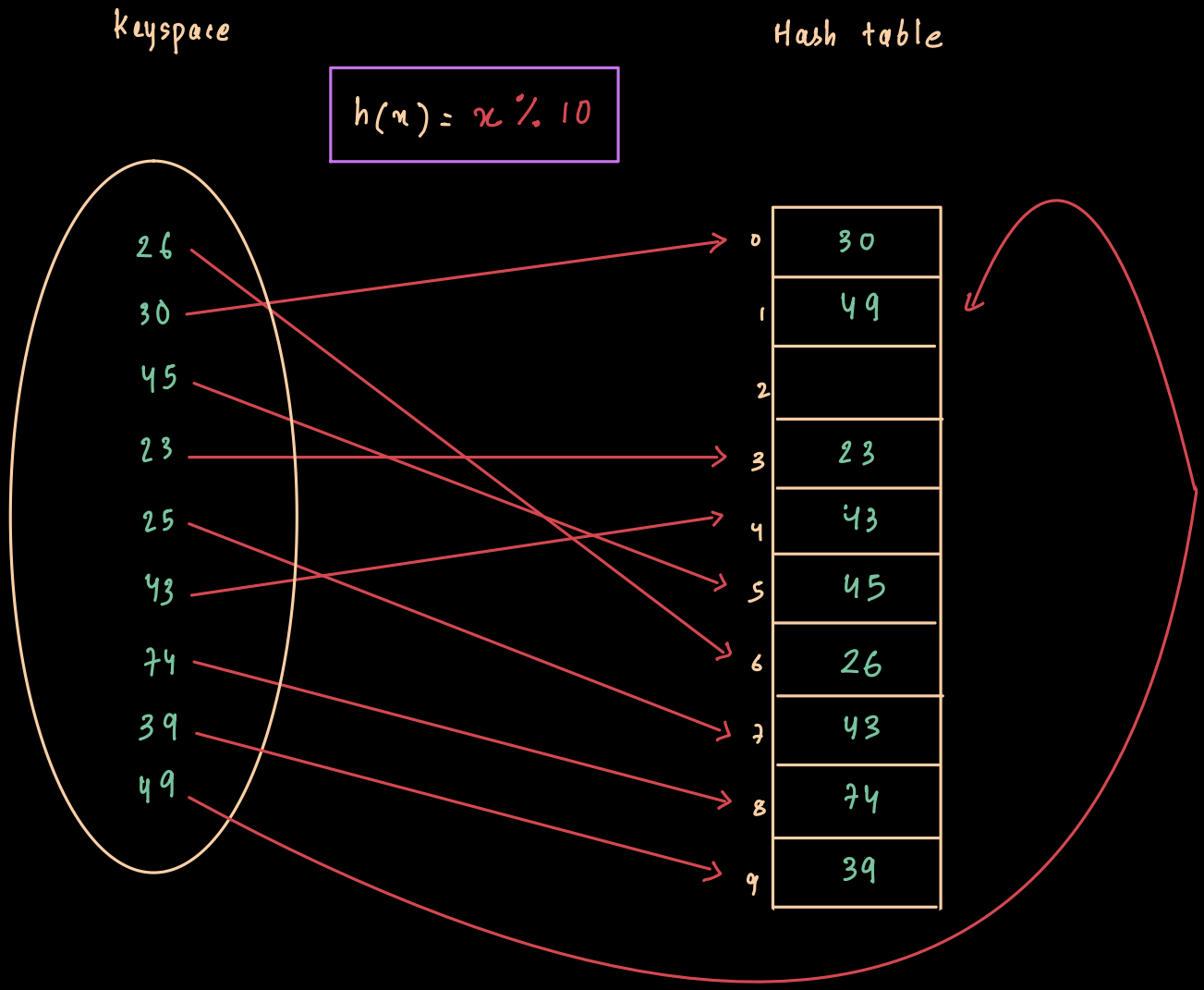
367. Let's code Chaining

```
int hash ( int key ) {
    return key % 10 ;
}
void Insert ( struct Node *H[], int key ) {
    int index = hash ( key ) ;
    SortedInsert ( &H[index], key ) ;      // Insert sorted element into a Linked list
```

```
}

int main(){
        struct Node *HT[10];
        int i;
        for(i=0; i<10; i++){
                HT[i] = NULL;
        }
        Insert(HT,12);
        Insert(HT,22);
        Insert(HT,42);

        // let's perform search
        struct Node *temp;
        temp = Search(HT[hash(22)],22);
        printf("%d ", temp->data);

        return 0;
}
```

(CLOSED HASHING) :

368. Linear Probing



* key : 49 → There's no space at index 9
So, it will check index 0 again
and the get stored wherever free.

* $h'(x) = (h(x) + f(i)) \% 10$ where $f(i) = i$
$i = 0, 1, 2, \ldots$

Ex: $h'(25) = (h(25) + f(0)) \% 10$

$= (5 + 0) \% 10$

$= 5$

if collision

$h'(25) = (h(25) + f(1)) \% 10$

$= (5 + 1) \% 10$

$= 6$

if collision

$h'(25) = (h(25) + f(2)) \% 10$

$= (5 + 2) \% 10$

$= 7$

If there is collision, then store key in the next free available space.

* Time complexity : little time taking

Analysis :

Loading factor (λ) $= \frac{n}{size}$

ie : $\lambda = \frac{9}{10} = 0.9$

* $\lambda \leq 0.5$ should be less than 0.5, so that free spaces are there for effieient searching.

* Avg. Successful search

$$t = \frac{1}{\lambda} \ln\left(\frac{1}{1-\lambda}\right)$$

* Avg. Successful search

$$t = \frac{1}{1-\lambda}$$

* Drawbacks : 1) Need to keep half of table vacant, that is wastage of space

2) lot of keys might get stored one after another, forming a cluster (Primary clustering)

* Deletion : Solution for deletion is not that simple b coz it needs rearranging all the elements again

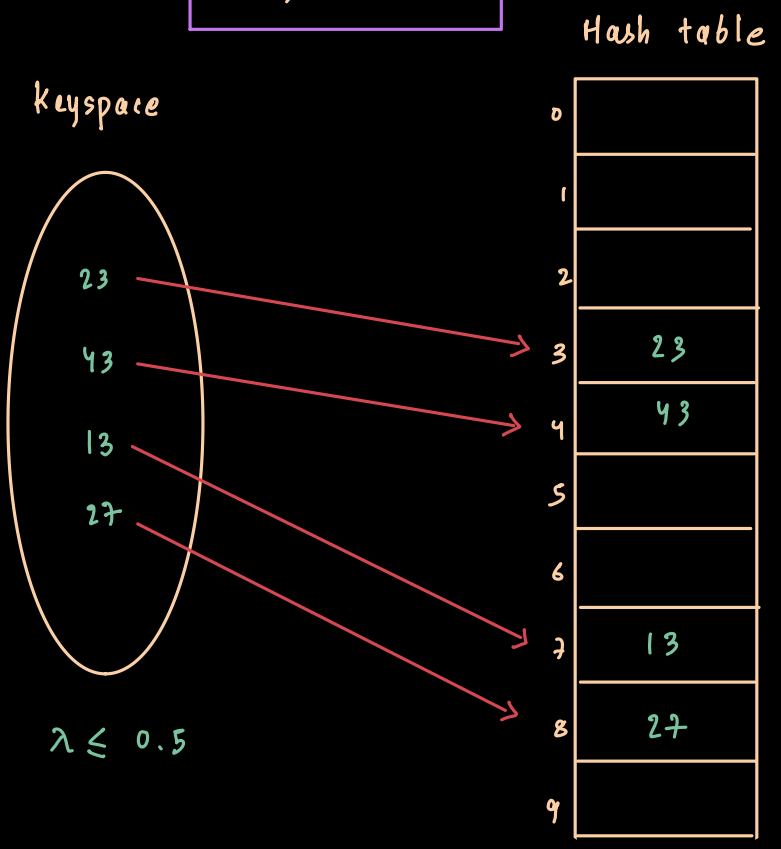
Better approach is DELETION & then REHASHING all keys again.

CODE :

```c
#include <stdio.h>
#define SIZE 10

int hash(int key) {
    return key % SIZE;
}

int probe(int H[], int key) {
    int index = hash(key);
    int i = 0;
    while(H[(index + i) % SIZE] != 0)
        i++;
    return (index + i) % SIZE;
}

void Insert(int H[], int key) {
    int index = hash(key);
    if(H[index] != 0)
        index = probe(H, key);
    H[index] = key;
}

int Search(int H[], int key) {
    int index = hash(key);
    int i = 0;
    while(H[(index + i) % SIZE] != key) {
        if(H[(index + i) % SIZE] == 0)
            return -1;  // not found
        i++;
    }
    return (index + i) % SIZE;
}

void Display(int H[]) {
    for(int i = 0; i < SIZE; i++) {
        printf("[%d] : %d\n", i, H[i]);
    }
}

int main() {
    int HT[SIZE] = {0};

    Insert(HT, 5);
    Insert(HT, 25);
    Insert(HT, 15);
    Insert(HT, 35);
    Insert(HT, 95);

    Display(HT);

    int key = 25;
    int index = Search(HT, key);
    if(index != -1)
        printf("Key %d found at index %d\n", key, index);
    else
        printf("Key %d not found\n", key);

    return 0;
}
```

370. Quadratic Probing

$h(x) = x \% 10$



$\lambda \leq 0.5$

$h'(23) = (h(23) + f(0)) \% 10$

$= 3$

$h'(43) = (h(43) + f(0)) \% 10$

$= 3$ (collision)

$(h(43) + f(1)) \% 10$

$= (3 + 1^2) \% 10$

$= 4$

$h'(13) = (h(13) + f(2)) \% 10$

$= (3 + 2^2) \% 10$

$= 7$

* $h'(x) = (h(x) + f(i)) \% 10$ where $f(i) = i^2$

$i = 0, 1, 2, \ldots$

* avg. successful search : $t = \frac{-\log_e(1-\lambda)}{\lambda}$

* avg. unsuccessful search : $t = \frac{1}{1-\lambda}$

371. Double Hashing

*

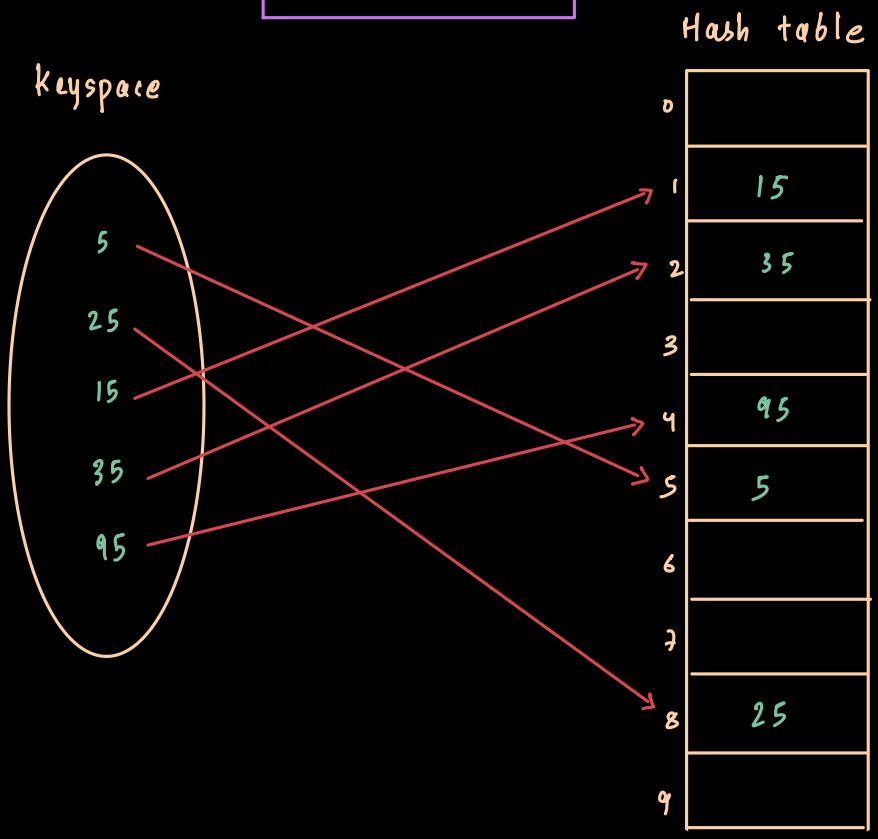
$h_1(x) = x \,\%\, 10$

$h_2(x) = R - (x \,\%\, R)$ → R = greatest prime no. less than size of hash table

$h'(x) = (h_1(x) + i * h_2(x)) \,\%\, 10$

where $i = 0, 1, 2, \ldots$

$h_1(x) = x \,\%\, 10$



Here, $h_1(x) = x \,\%\, 10$

$h_2(x) = 7 - (x \,\%\, 7)$ $(R = 7)$

$h'(x) = (h_1(x) + i * h_2(x)) \,\%\, 10$

where $i = 0, 1, 2, \ldots$

- $h'(5) = 5 \% 10$
  $= 5$

- $h'(25) = (5 + 0 * h_2(x)) \% 10$
  $= 5$ (collision)
  $(5 + 1 * 3) \% 10$
  $= (5 + 3) \% 10$
  $= 8$

$h_2(25) = 7 - (25 \% 7)$
$= 7 - (4)$
$= 3$

- $h'(15) = (5 + 1 * 6) \% 10$
  $= 11 \% 10$
  $= 1$

$h_2(15) = 7 - (15 \% 7)$
$= 7 - 1$
$= 6$

- $h'(35) = (5 + 1 * 7) \% 10$
  $= 12 \% 10$
  $= 2$

$h_2(35) = 7 - (35 \% 7)$
$= 7$

- $h'(95) = (5 + 1 * 3) \% 10$
  $= 8$ (collision)
  $(5 + 2 * 3) \% 10$
  $= 11 \% 10$
  $= 1$ (collision)
  $(5 + 3 * 3) \% 10$
  $= 4$

$h_2(95) = 7 - (95 \% 7)$
$= 7 - 4$
$= 3$